Rust vs. C++ for CPU and GPU Performance: A Repository-Based Benchmark Analysis

Aditya Raut
CSCI 693 Research Methods in Computer Science

1 Introduction

High-performance computing continues to rely heavily on low-level languages that offer explicit control over memory layout, optimization behavior, and hardware access. C++ remains the dominant implementation language in many performance-critical CPU and GPU systems because of mature compilers, direct CUDA integration, and a large body of existing libraries. Rust, however, has emerged as a strong alternative for systems programming by combining ahead-of-time compilation with a memory-safe ownership model and strong concurrency guarantees.

The core question for practitioners is whether Rust can offer comparable performance to C++ while reducing safety risks and improving maintainability. Prior work suggests that Rust often approaches C++ performance on CPU workloads and may be viable for GPU-related development when paired with CUDA back ends or driver bindings [1–6]. The repository associated with this study publishes raw benchmark outputs and derived summaries for a controlled Rust versus C++ comparison across both CPU and GPU tasks. This draft paper uses those committed raw artifacts directly, without rerunning code, to assemble the initial conference-style manuscript and summarize the current empirical findings.

2 Related Work

Recent literature has examined Rust's runtime behavior, compiler optimizations, and applicability to high-performance workloads. Zhang *et al.* study Rust's runtime performance characteristics and provide a broader view of the language's execution behavior in systems contexts [6]. Ivanov compares Rust with C++ on common routines and reports that Rust can be competitive on many optimized CPU tasks [2]. Martins *et al.* extend the conversation to high-performance computing benchmarks and show that the gap between Rust and established HPC toolchains is narrowing [4].

On the GPU side, Bychkov and Nikolskiy examine Rust as a language for GPU programming and discuss the importance of equivalent kernels and consistent memory management when evaluating performance [1]. Köpcke *et al.* emphasize the broader research interest in safe GPU systems programming through the Descend language [3]. Veytsman *et al.* contribute an applied case study in computational physics, arguing that Rust can reduce classes of memory-related defects while still preserving strong execution performance [5]. Taken together, these studies motivate a unified CPU and GPU comparison grounded in the same project artifacts and evaluation pipeline.

3 Experimental Design

This draft is based on the committed raw results and summary artifacts stored in the public project repository.[1] The repository includes CPU and GPU implementations, exported benchmark CSV files, statistical summaries, and a compact safety metrics report. No new execution was performed for this draft; all numerical claims in the current manuscript are drawn from the published repository artifacts under `results/raw`.

The benchmark suite covers CPU and GPU workloads relevant to numerical and systems-oriented computing:

- CPU: binary search, lower bound search, quicksort, mergesort, dense matrix multiplication, FFT, hash table insertion and lookup, n-body simulation, and parallel reduction.
- GPU: CUDA matrix multiplication, CUDA reduction, and CUDA softmax.

The summary CSV reports mean runtime, standard deviation, confidence intervals, percentage overhead, t-statistics, p-values, and effect sizes for each benchmark. The repository metadata also include coarse safety-oriented counts such as lines of code and recorded Rust `unsafe` block usage.

4 Results

4.1 Representative Benchmark Outcomes

Table 1 presents a compact set of representative results drawn from the repository's raw summary table. Negative overhead indicates that Rust is faster than C++, while positive overhead indicates slower Rust runtimes.

Table 1: Representative results from committed raw summary data

Benchmark	C++ (ms)	Rust (ms)	Overhead
binary_search	7.20	2.28	-68.3%
quicksort_std_sort	49.70	9.62	-80.6%
mergesort_recursive	63.30	53.55	-15.4%
hashmap_int_insert	3.78	1.08	-71.3%
matmul_naive	14.23	18.57	+30.5%
fft_forward	0.93	1.22	+30.9%
parallel_reduction_rayon	0.03	0.10	+192.2%
cuda_matmul_tiled	0.0018	0.0002	-86.5%
cuda_reduction	0.0633	0.0002	-99.6%
cuda_softmax	0.0012	0.0017	+37.7%

4.2 CPU Findings

The CPU results do not support a simple conclusion that one language dominates all workloads. Instead, the committed summaries suggest a benchmark-dependent outcome. Rust shows substantial advantages on binary search, lower bound, integer hash map insertion, standard-library quicksort, and recursive mergesort. For

[1] https://github.com/adityaanilraut/Rust-vs-Cpp-for-CPU-and-GPU-Performance

example, the reported binary search result is 2.2820 ms for Rust versus 7.1952 ms for C++, while the standard-sort quicksort configuration shows 9.6229 ms for Rust versus 49.7041 ms for C++.

At the same time, C++ retains clear advantages on naive dense matrix multiplication, FFT forward and round-trip runs, and the Rayon-based parallel reduction configuration. The largest CPU disadvantage for Rust in the current summaries is the parallel reduction benchmark, where the Rust configuration reports a 192.23% overhead relative to C++. The n-body benchmarks are close to parity, with only 0.26% and 0.57% overhead in the reported summaries.

These mixed CPU outcomes suggest that implementation strategy and library/runtime choices may be as important as language choice. In particular, the difference between Rust's strong results on sequential and library-backed routines and its weak result on the Rayon reduction configuration suggests that abstraction overhead, scheduling behavior, or implementation details may materially affect comparative performance.

4.3 GPU Findings

The committed GPU summary data indicate strong Rust results for the matrix multiplication and reduction kernels, with reported overheads of -86.52% and -99.62% respectively. In contrast, the softmax benchmark reports a +37.7% Rust overhead with a p-value of 0.7781, which does not indicate a statistically significant difference in the current raw summary.

The GPU entries also show noticeably higher coefficients of variation on some tasks, especially softmax, than the more stable CPU measurements. This indicates that the GPU findings should be interpreted more cautiously than the strongest CPU results. Still, based on the committed artifacts alone, the current raw data suggest that Rust can be competitive and in some cases superior on the evaluated CUDA-oriented kernels.

4.4 Safety and Code Metrics

The repository's safety metrics report the following line counts: 739 lines for C++ CPU code, 709 lines for C++ GPU code, 731 lines for Rust CPU code, and 141 lines for Rust GPU code. The report also records zero Rust `unsafe` blocks across the measured CPU and GPU code paths, with an aggregate Rust-to-C++ line count ratio of 0.6. Those figures are reproduced in Table 2.

Table 2: Safety and code metrics from committed repository metadata

Metric	Value
C++ CPU LOC	739
C++ GPU LOC	709
Rust CPU LOC	731
Rust GPU LOC	141
Rust/C++ LOC ratio	0.6
Recorded Rust unsafe blocks	0

These metadata do not by themselves prove lower maintenance cost, but they do support the argument that the repository's Rust implementations achieve their current results without recorded `unsafe` usage in the measured paths.

5 Discussion

The committed raw data support three preliminary observations. First, Rust is clearly viable for high-performance CPU work and can outperform C++ on several workloads when the underlying implementation strategy aligns well with the optimizer and standard library behavior. Second, the GPU results indicate that Rust-based CUDA workflows can be competitive in practice, at least for the kernels represented in the committed repository outputs. Third, the safety metrics suggest that these results were obtained without resorting to explicit `unsafe` blocks in the Rust benchmark paths, reinforcing the broader claim that safety and performance need not be mutually exclusive.

However, the results are not uniformly favorable to Rust. C++ remains stronger in several numerically intensive or explicitly parallel CPU cases, especially FFT and the reported Rayon reduction benchmark. This means the current data are more consistent with a nuanced trade-off than with a universal language winner. A strong final version of the paper should therefore emphasize workload sensitivity and implementation details rather than making a blanket claim about language superiority.

6 Threats to Validity

This draft is intentionally limited to already-published repository artifacts and does not rerun or independently verify the benchmark suite. As a result, the present manuscript should be understood as a repository-grounded draft rather than a fully revalidated final paper. In addition, the benchmark set focuses on a small collection of kernels and does not represent all HPC or AI workloads. Some GPU measurements also exhibit high variance, which weakens confidence in broad generalization. Finally, line count and `unsafe` block counts are only partial proxies for safety and maintainability; they should be interpreted as supporting metadata rather than conclusive evidence.

7 Conclusion

Using the committed raw benchmark and metadata artifacts in the project repository, this draft finds that Rust is competitive with C++ across a diverse set of CPU and GPU workloads and outperforms C++ on several reported tasks. The strongest Rust results in the current summaries appear in search, sorting, hash map insertion, and two of the three reported GPU kernels, while C++ retains advantages in FFT, naive matrix multiplication, and the reported Rayon reduction case. The current repository metadata also suggest that these outcomes were achieved without recorded Rust `unsafe` blocks in the measured code paths.

The next revision of this paper should add final figures, expand the methodology discussion, and refine the interpretation of the GPU results using the already-published raw repository artifacts. Even in its current draft form, the evidence supports the broader claim that Rust is a credible language for performance-sensitive CPU and GPU programming when safety and maintainability are also part of the design objective.

References

[1] A. Bychkov and V. Nikolskiy. 2022. Rust Language for GPU Programming. In *Supercomputing – RuSCDays 2022*. Lecture Notes in Computer Science, Vol. 13708. Springer, 522–532. doi:10.1007/978-3-031-22941-1_38

[2] N. Ivanov. 2022. Is Rust C++-fast? Benchmarking System Languages on Everyday Routines. *arXiv preprint arXiv:2209.09127* (2022).
[3] B. Köpcke, S. Gorlatch, and M. Steuwer. 2023. Descend: A Safe GPU Systems Programming Language. *arXiv preprint arXiv:2305.03448* (2023).
[4] E. M. Martins, L. G. Faé, R. B. Hoffmann, L. S. Bianchessi, and D. Griebler. 2025. NPB-Rust: NAS Parallel Benchmarks in Rust. *arXiv preprint arXiv:2502.15536* (2025).
[5] W. Veytsman, S. Zhai, C. Ding, and A. B. Sefkow. 2024. Rewrite it in Rust: A Computational Physics Case Study. *arXiv preprint arXiv:2410.19146* (2024).
[6] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. 2022. Towards Understanding the Runtime Performance of Rust. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Rochester, MI, USA. doi:10.1145/3551349.3559494